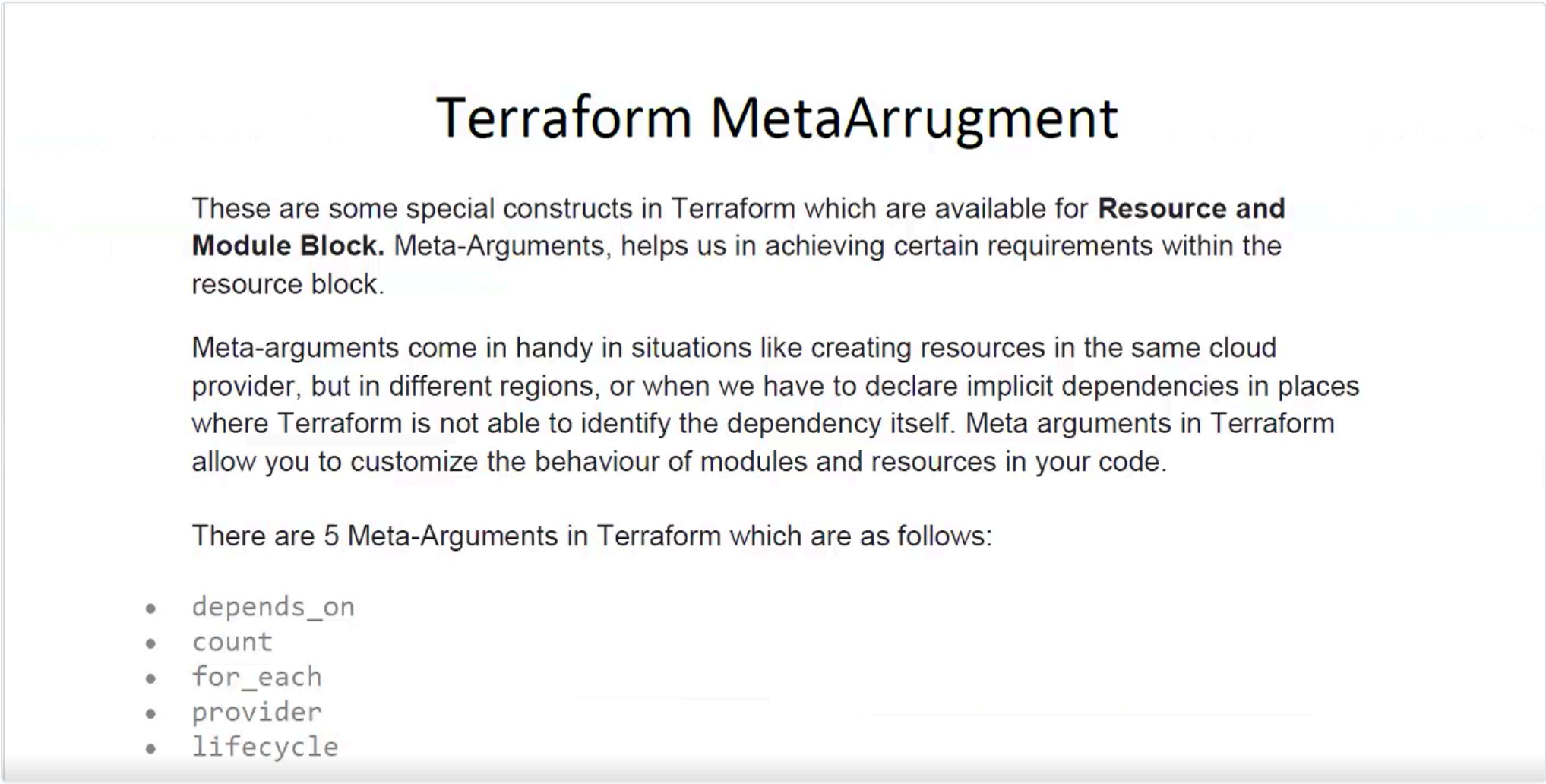
Screenshots — Terraform Meta-Arguments

Lecture 10 · 62 shots, chronological · source stills from the session (2026-10-07 12:52 – 15:05)

01 / 62 Slide — meta-arguments for Resource and Module blocks; the 5: depends_on, count, for_each, provider, lifecycle `12:52:33`



Terraform meta-arguments are special, built-in configuration arguments that control how HashiCorp Terraform manages, scales, and deploys infrastructure resources. While standard arguments define the properties of a resource (like a server's size or name), meta-arguments change how Terraform interacts with the resource block itself. They are universally supported across all providers (like AWS, GCP, or Azure). YouTube · Ned in the Cloud +2

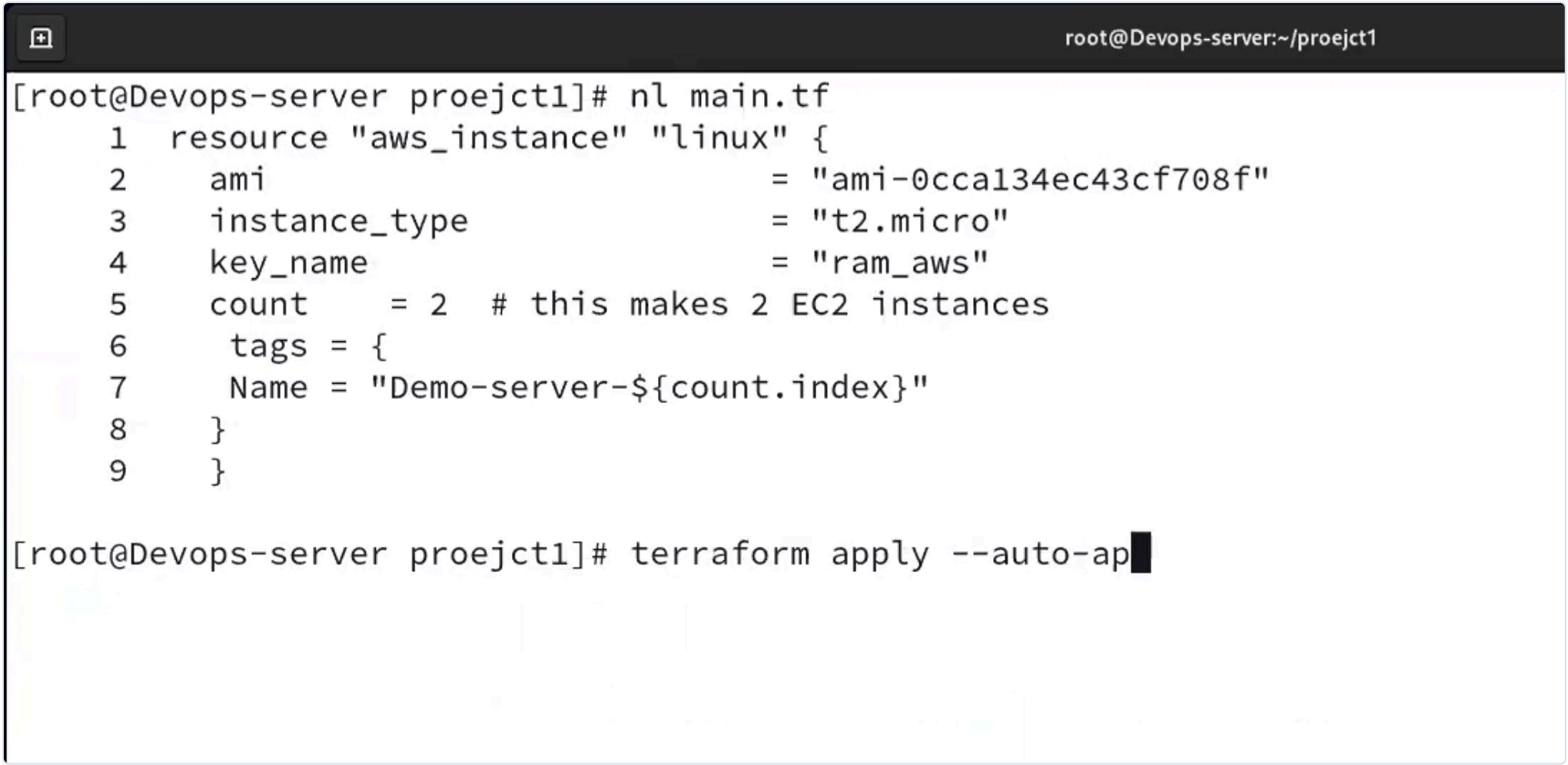
root@Devops-server:~/proejct1 — vim main.tf

```
resource "aws_instance" "linux" {
  ami                         = "ami-0cca134ec43cf708f"
  instance_type               = "t2.micro"
  key_name                    = "ram_aws"
  count     = 2  # this makes 2 EC2 instances
   tags = {
   Name = "Demo-server-${count.index}"
  }
  }

~
~
~
~
```

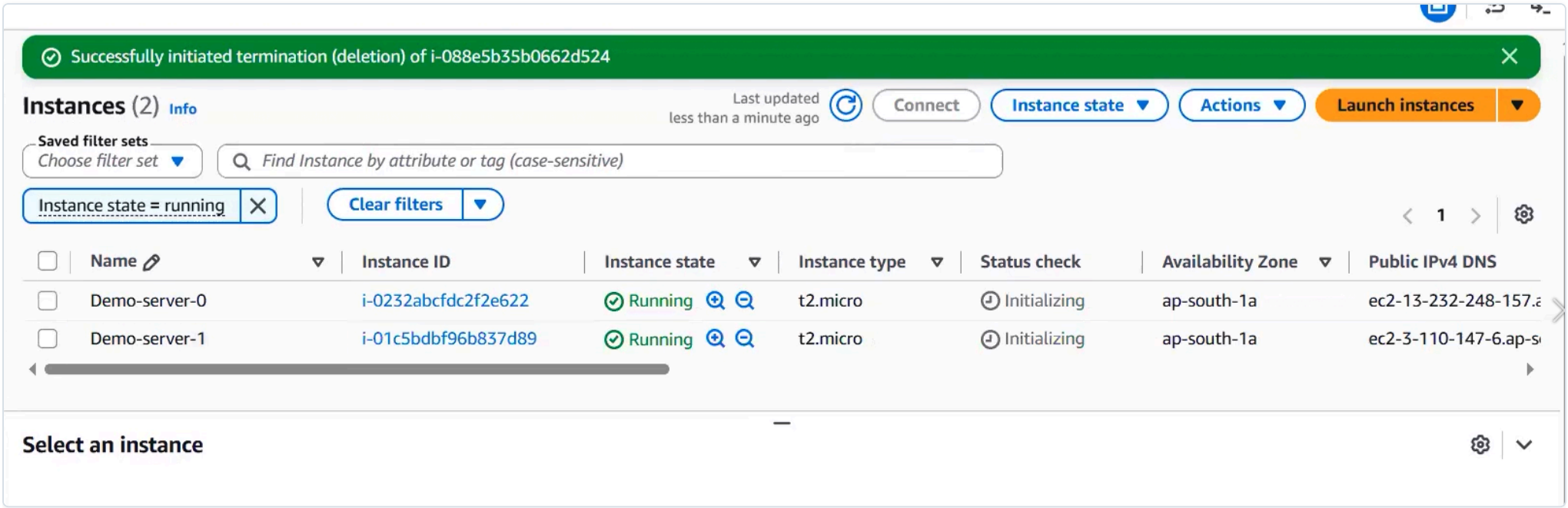
1. `count`

- **Purpose**: Creates a specific number of near-identical infrastructure objects from a single block.
- **Behavior**: Accepts a numeric integer value.
- **Tracking**: Uses a zero-indexed tracking system (`count.index`) to distinguish between instances.
- **Limitation**: Adding or removing items from the middle of a list indexed by `count` can force Terraform to destroy and recreate unrelated resources. YouTube · DevOps Avenue +4


root@Devops-server:~/proejct1
[root@Devops-server proejct1]# nl main.tf
1 resource "aws_instance" "linux" {
2 ami = "ami-0cca134ec43cf708f"
3 instance_type = "t2.micro"
4 key_name = "ram_aws"
5 count = 2 # this makes 2 EC2 instances
6 tags = {
7 Name = "Demo-server-${count.index}"
8 }
9 }
[root@Devops-server proejct1]# terraform apply --auto-ap

```
Plan: 2 to add, 0 to change, 0 to destroy.
aws_instance.linux[0]: Creating...
aws_instance.linux[1]: Creating...
```

Console — Demo-server-0 and Demo-server-1 running

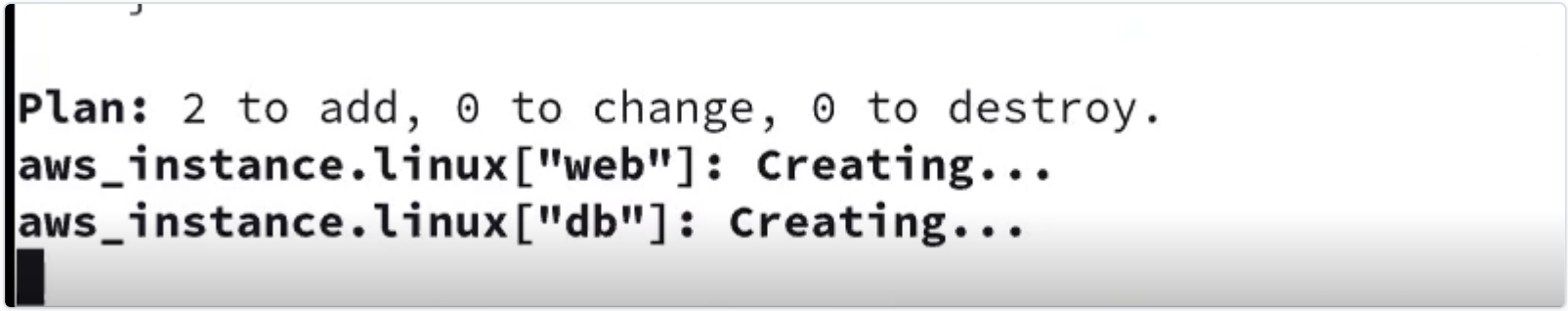


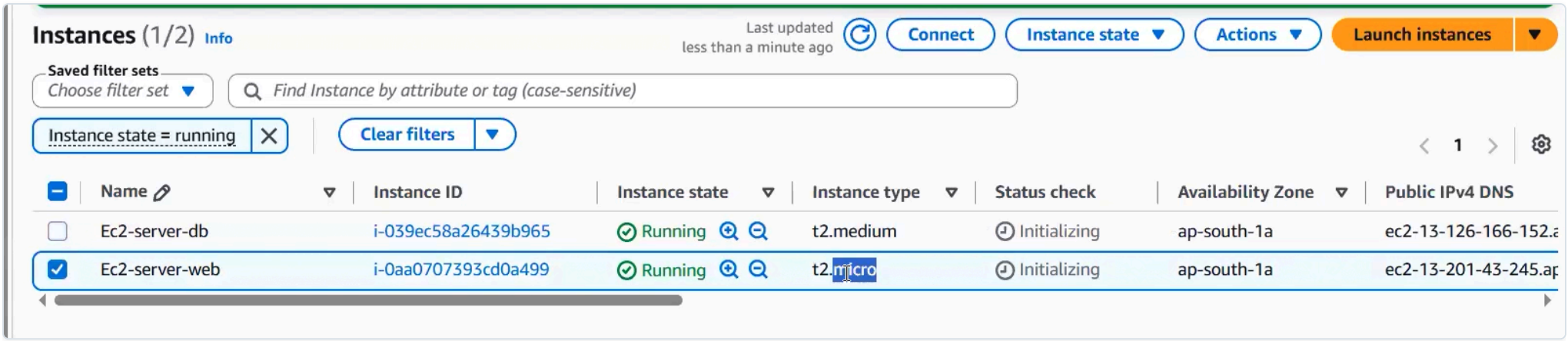

for_each
"Loop over a list or map."

root@Devops-server:~/proejct1 — vim main.tf

```
resource "aws_instance" "linux" {
  ami                         = "ami-0cca134ec43cf708f"
  for_each = {
     web = "t2.micro"
     db = "t2.medium"
}
  instance_type               = each.value
  key_name                    = "ram_aws"
   tags = {
   Name = "Ec2-server-${each.key}"
  }
}
```

Plan 2 to add — `linux["web"]` and `linux["db"]`

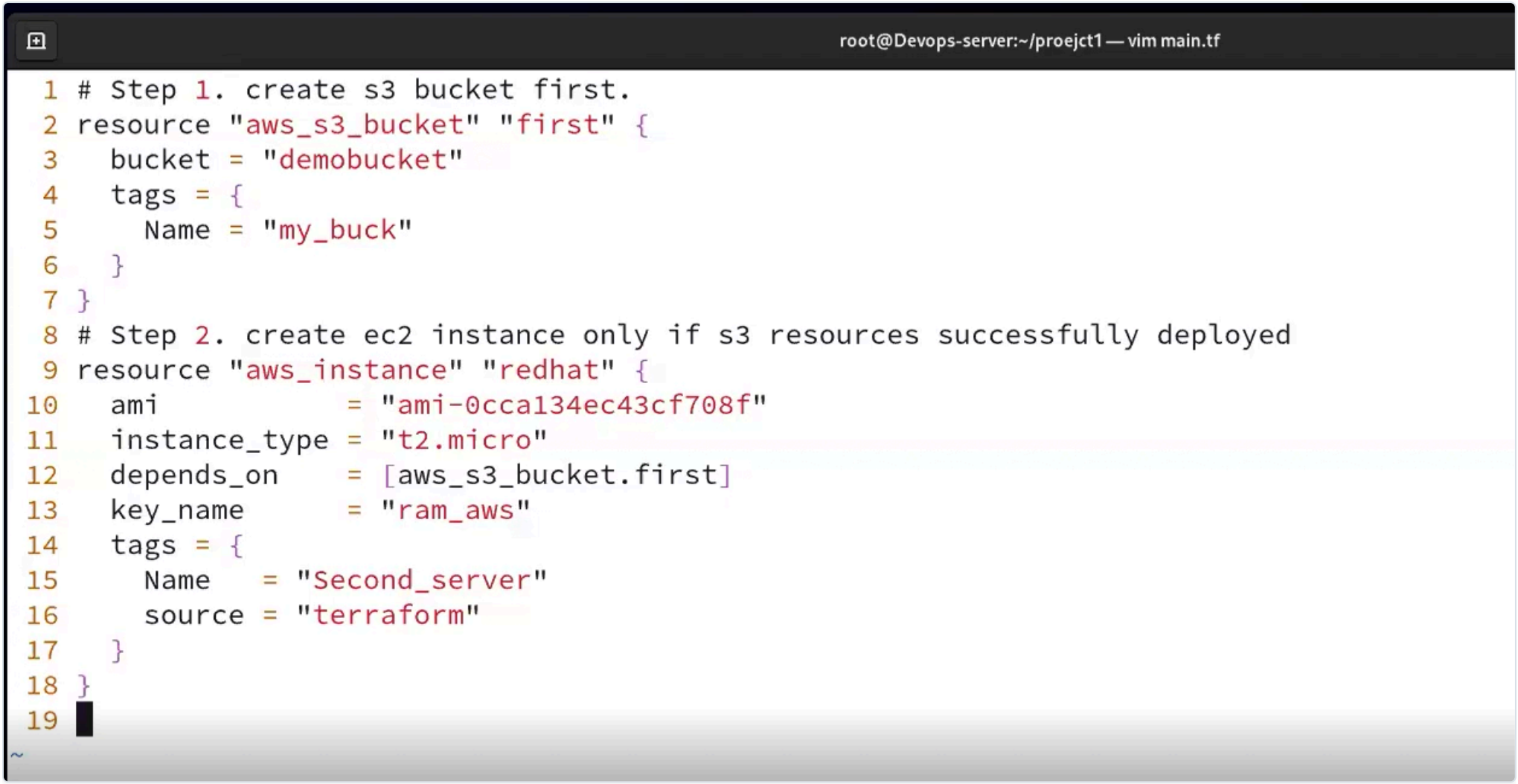



Instances (1/2) Info
Last updated less than a minute ago
Connect
Instance state
Actions
Launch instances
Saved filter sets
Choose filter set
Find Instance by attribute or tag (case-sensitive)
Instance state = running
Clear filters
1
Name
Instance ID
Instance state
Instance type
Status check
Availability Zone
Public IPv4 DNS
Ec2-server-db
i-039ec58a26439b965
Running
t2.medium
Initializing
ap-south-1a
ec2-13-126-166-152.a
Ec2-server-web
i-0aa0707393cd0a499
Running
t2.micro
Initializing
ap-south-1a
ec2-13-201-43-245.ap

3. depends_on

- **Purpose**: Enforces an explicit deployment order when Terraform cannot naturally discover infrastructure dependencies.
- **Behavior**: Accepts a list of target resources or modules.
- **Usage**: Best used as a last resort, as overusing it can overcomplicate dependency graphs and slow down parallel deployments. HashiCorp Developer +4

depends_on Wait until another resource is ready


root@Devops-server:~/proejct1 — vim main.tf
Step 1. create s3 bucket first.
resource "aws_s3_bucket" "first" {
bucket = "demobucket"
tags = {
Name = "my_buck"
}
}
Step 2. create ec2 instance only if s3 resources successfully deployed
resource "aws_instance" "redhat" {
ami = "ami-0cca134ec43cf708f"
instance_type = "t2.micro"
depends_on = [aws_s3_bucket.first]
key_name = "ram_aws"
tags = {
Name = "Second_server"
source = "terraform"
}
}

```
Plan: 2 to add, 0 to change, 0 to destroy.
aws_s3_bucket.first: Creating...

 Error: creating S3 Bucket (demobucket): operation error S3: CreateBucket, https response error StatusCode: 409, Reques
tID: XG25W5ZGDJQXBH0D, HostID: kfHq92tYba6tVqvSSgVH1eCRcy69GN8vkYSOCTI2vfxrspuIMHq/7md4u8BVsBny1KBEpBEG9Io=, BucketAlrea
dyExists: The requested bucket name is not available. The bucket namespace is shared by all users of the system. Please
select a different name and try again.

   with aws_s3_bucket.first,
   on main.tf line 2, in resource "aws_s3_bucket" "first":
    2: resource "aws_s3_bucket" "first" {
```

4. `provider`

- **Purpose**: Assigns a non-default provider configuration or region to a specific block.
- **Behavior**: Dictates which cloud account or infrastructure region manages the resource.
- **Usage**: Ideal when setting up multi-region networking or deploying cross-account components. Medium · Sayali Shewale +3

root@Devops-server:~/proejct1 — vim main.tf

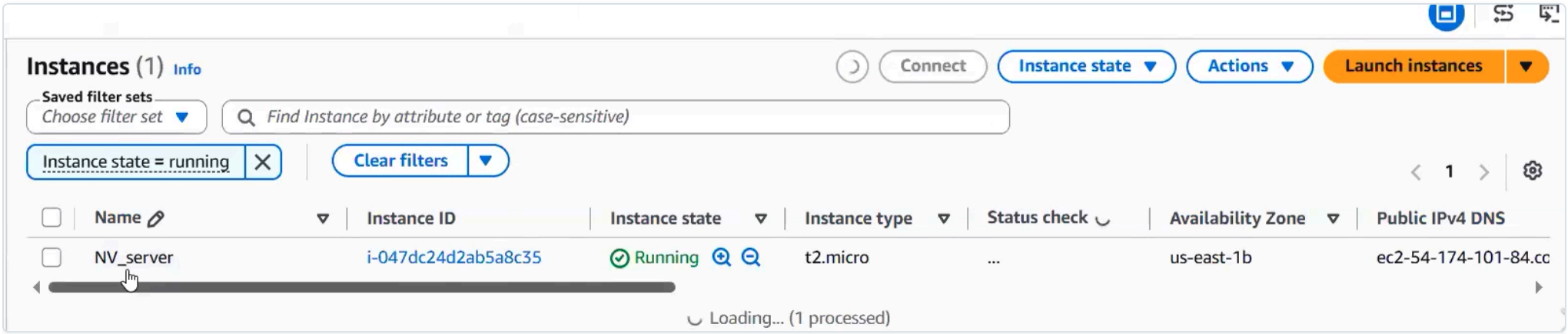
```
 5 provider "aws" {
 6 region = "us-east-1"
 7 alias = "NV"
 8 }
 9 resource "aws_instance" "linux" {
10   ami           = "ami-0cca134ec43cf708f"
11   instance_type = "t2.micro"
12   key_name      = "ram_aws"
13   provider = aws.mumbai
14   tags = {
15     Name = "Mumbai_server"
16   }
17 }
18 resource "aws_instance" "linux1" {
19   ami           = "ami-03a6eaae9938c858c"
20   instance_type = "t2.micro"
21   key_name      = "ram_nv"
22   provider = aws.NV
23   tags = {
24     Name = "NV_server"
25   }
26 }
```

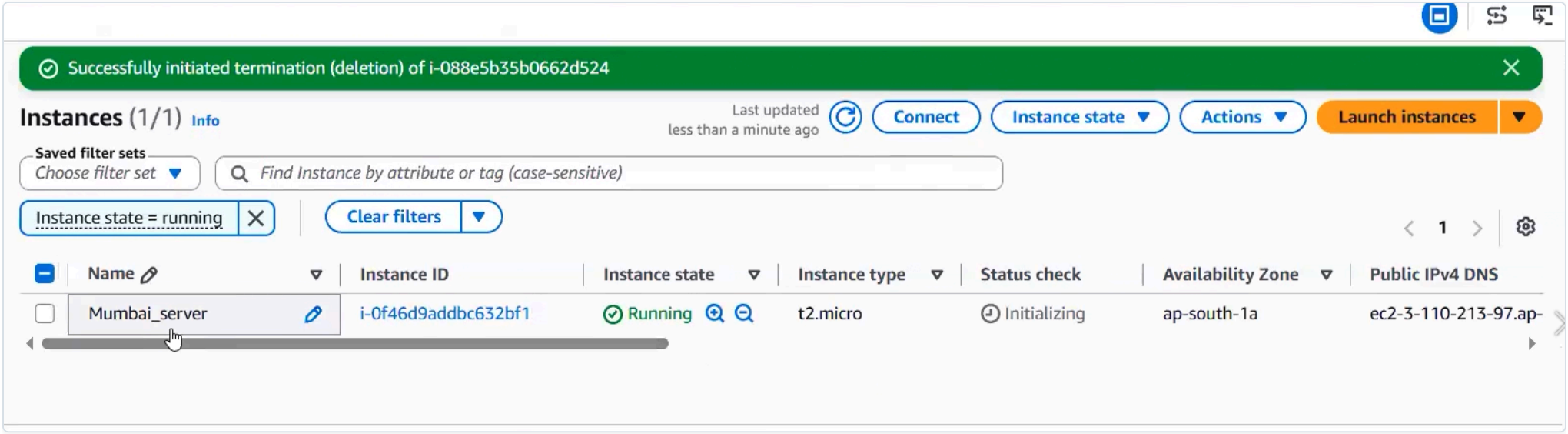
```
provider "aws" {
region = "ap-south-1"
alias = "mumbai"
}

provider "aws" {
region = "us-east-1"
alias = "NV"
}
resource "aws_instance" "linux" {
  ami           = "ami-0cca134ec43cf708f"
  instance_type = "t2.micro"
  key_name      = "ram_aws"
  provider = aws.mumbai
  tags = {
    Name = "Mumbai_server"
  }
}
resource "aws_instance" "linux1" {
  ami           = "ami-03a6eaae9938c858c"
  instance_type = "t2.micro"
  key_name      = "ram_nv"
  provider = aws.NV
```

```
Plan: 2 to add, 0 to change, 0 to destroy.
aws_instance.linux: Creating...
aws_instance.linux1: Creating...
aws_instance.linux: Still creating... [00m10s elapsed]
aws_instance.linux1: Still creating... [00m10s elapsed]
aws_instance.linux: Still creating... [00m20s elapsed]
aws_instance.linux1: Still creating... [00m20s elapsed]
aws_instance.linux: Still creating... [00m30s elapsed]
aws_instance.linux1: Still creating... [00m30s elapsed]
aws_instance.linux1: Creation complete after 32s [id=i-047dc24d2ab5a8c35]
aws_instance.linux: Still creating... [00m40s elapsed]
aws_instance.linux: Creation complete after 43s [id=i-0f46d9addbc632bf1]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
```


Instances (1) Info
Connect
Instance state
Actions
Launch instances
Saved filter sets
Choose filter set
Find Instance by attribute or tag (case-sensitive)
Instance state = running
Clear filters
1
Name
Instance ID
Instance state
Instance type
Status check
Availability Zone
Public IPv4 DNS
NV_server
i-047dc24d2ab5a8c35
Running
t2.micro
...
us-east-1b
ec2-54-174-101-84.cc
Loading... (1 processed)


Successfully initiated termination (deletion) of i-088e5b35b0662d524
Instances (1/1) Info
Last updated less than a minute ago
Connect
Instance state
Actions
Launch instances
Saved filter sets
Choose filter set
Find Instance by attribute or tag (case-sensitive)
Instance state = running
Clear filters
1
Name
Instance ID
Instance state
Instance type
Status check
Availability Zone
Public IPv4 DNS
Mumbai_server
i-0f46d9addbc632bf1
Running
t2.micro
Initializing
ap-south-1a
ec2-3-110-213-97.ap-

5. `lifecycle`

- **Purpose**: A nested configuration block used to override Terraform's standard create, update, or destroy behaviors. YouTube · DevOps Avenue
- **Key Lifecycle Sub-arguments**:
  - `create_before_destroy` : Generates replacement infrastructure before deleting old parts to eliminate service downtime.
  - `prevent_destroy` : Rejects execution plans that attempt to delete critical resources like production databases.
  - `ignore_changes` : Overlooks manual external modifications made to specified resource properties.
  - `replace_triggered_by` : Mandates complete resource recreation if an external item or property shifts. HashiCorp Developer +2

root@Devops-server:~/proejct1 — vim main.tf

```
resource "aws_instance" "linux1" {
  ami           = "ami-0cca134ec43cf708f"
  instance_type = "t2.micro"
  key_name      = "ram_aws"
  tags = {
    Name = "Web_server"
  }

~
~
~
```

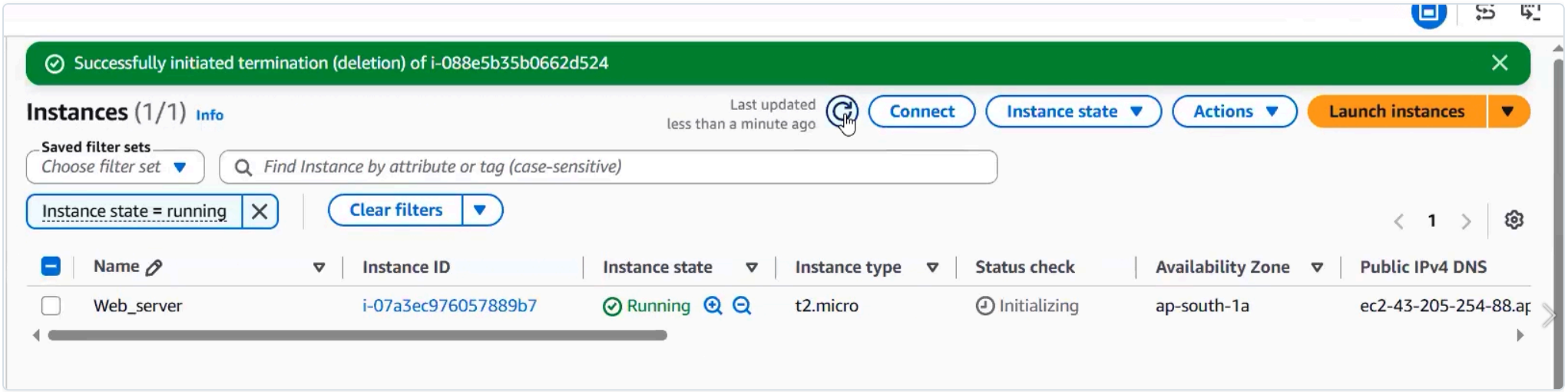
Adding `lifecycle { prevent_destroy = true }` (closing brace still missing)



root@Devops-server:~/proejct1 — vim main.tf

```
resource "aws_instance" "linux1" {
  ami           = "ami-0cca134ec43cf708f"
  instance_type = "t2.micro"
  key_name      = "ram_aws"
  tags = {
    Name = "Web_server"
  }
 lifecycle {
       prevent_destroy = true ##### to prevent destroy

}
}
~
~
~
~
~
```


Successfully initiated termination (deletion) of i-088e5b35b0662d524
Instances (1/1) Info
Last updated less than a minute ago
Connect
Instance state
Actions
Launch instances
Saved filter sets
Choose filter set
Find Instance by attribute or tag (case-sensitive)
Instance state = running
Clear filters
1
Name
Instance ID
Instance state
Instance type
Status check
Availability Zone
Public IPv4 DNS
Web_server
i-07a3ec976057889b7
Running
t2.micro
Initializing
ap-south-1a
ec2-43-205-254-88.ap

```
Plan: 1 to add, 0 to change, 0 to destroy.
aws_instance.linux1: Creating...
aws_instance.linux1: Still creating... [00m10s elapsed]
aws_instance.linux1: Still creating... [00m20s elapsed]
aws_instance.linux1: Still creating... [00m30s elapsed]
aws_instance.linux1: Creation complete after 33s [id=i-07a3ec976057889b7]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
[root@Devops-server proejct1]# terraform destroy --auto-approve
```

```
Plan: 0 to add, 0 to change, 1 to destroy.

  Error: Instance cannot be destroyed

    on main.tf line 1:
     1: resource "aws_instance" "linux1" {

  Resource aws_instance.linux1 has lifecycle.prevent_destroy set, but the plan calls for this resource to be destroyed.
  To avoid this error and continue with the plan, either disable lifecycle.prevent_destroy or reduce the scope of the
  plan using the -target option.

[root@Devops-server proejct1]#
```

root@Devops-server:~/proejct1 — vim main.tf

```
resource "aws_instance" "linux1" {
  ami           = "ami-0cca134ec43cf708f"
  instance_type = "t2.micro"
  key_name      = "ram_aws"
  tags = {
    Name = "Web_server"
  }
 lifecycle {
       prevent_destroy = true ##### to prevent destroy
        ignore_changes = [tags,instance_type ]

}
}
~
~
~
~
```

```
[root@Devops-server proejct1]# terraform apply --auto-approve
aws_instance.linux1: Refreshing state... [id=i-07a3ec976057889b7]

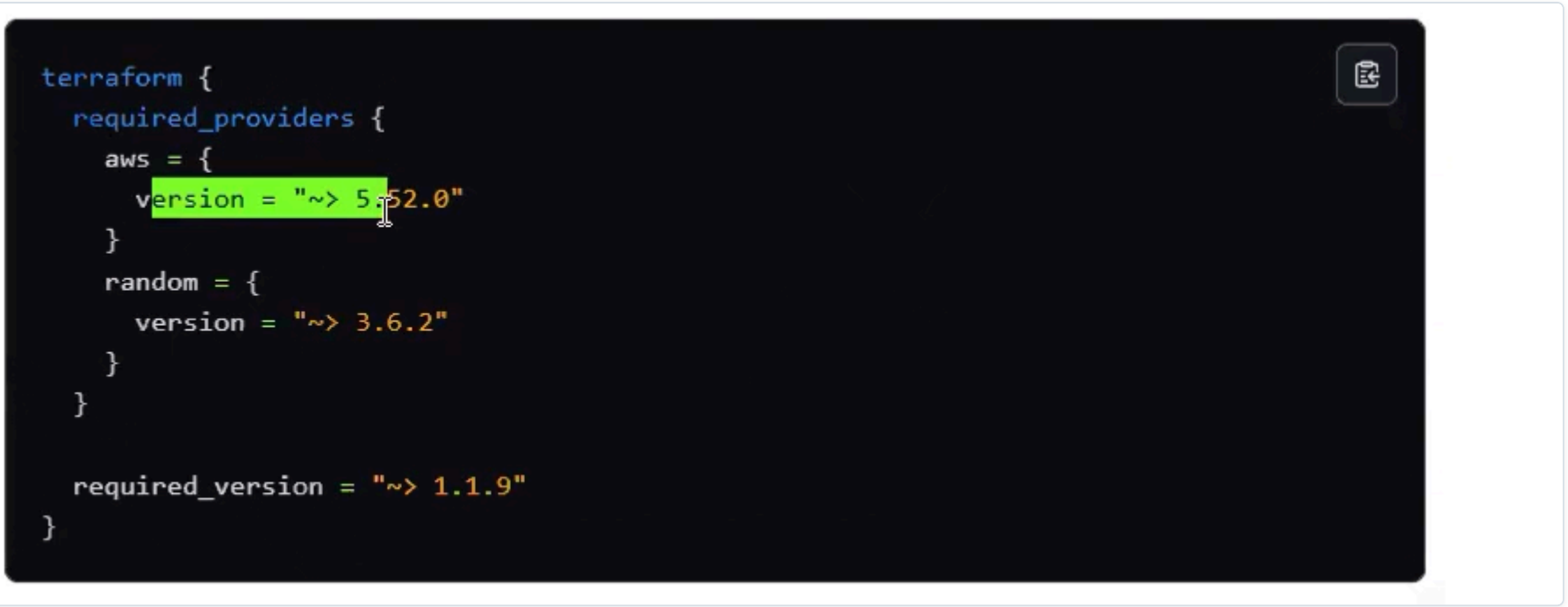
No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are
needed.
```

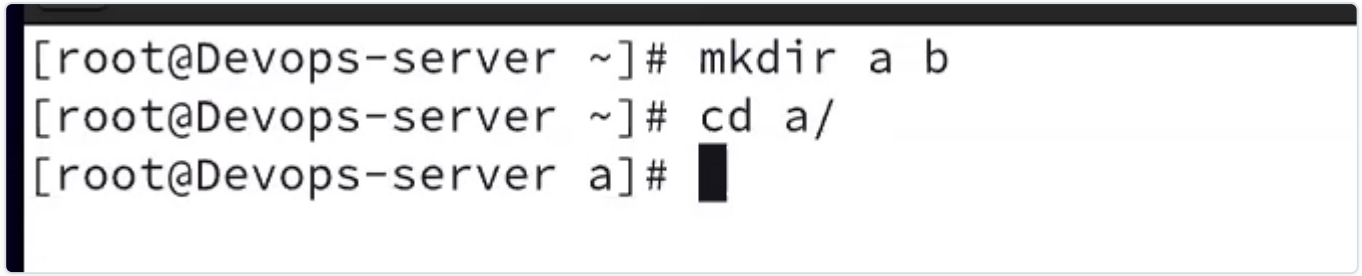

root@Devops-server:~/proejct1 — vim main.tf
resource "aws_instance" "linux1" {
ami = "ami-0cca134ec43cf708f"
instance_type = "t2.micro"
key_name = "ram_aws"
tags = {
Name = "Web_server"
}
lifecycle {
prevent_destroy = false##### to prevent destroy
ignore_changes = [tags,instance_type]
}
}

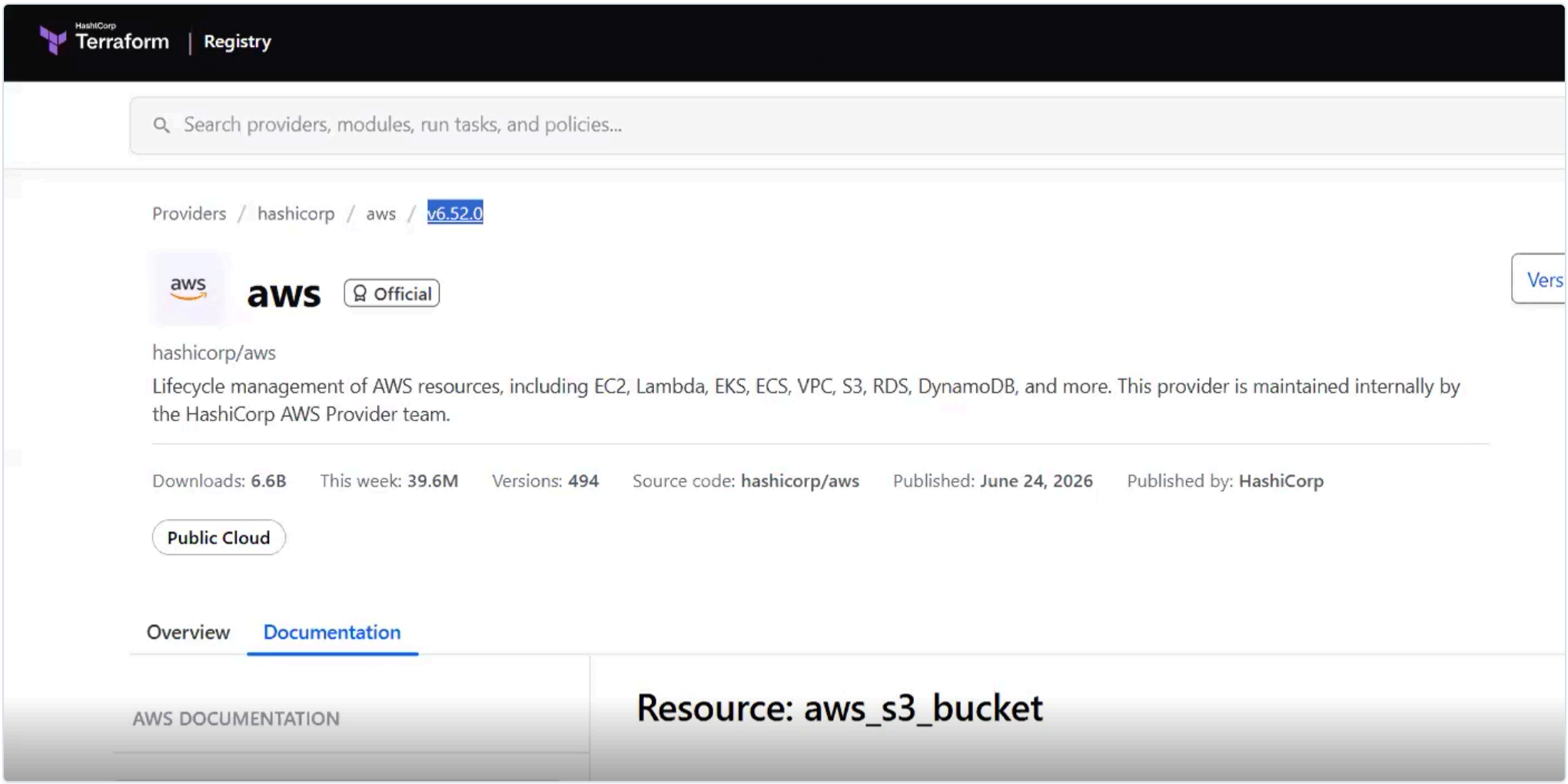
HashiCorp actively develops and maintains Terraform. To access new Terraform features you need to upgrade the version of Terraform that your configurations use. Set the `required_version` to control the version of Terraform that your configurations use and make updates more predictable.

In this tutorial, you will update an existing configuration to use the latest version of Terraform and learn how to manage different versions of Terraform within a team.

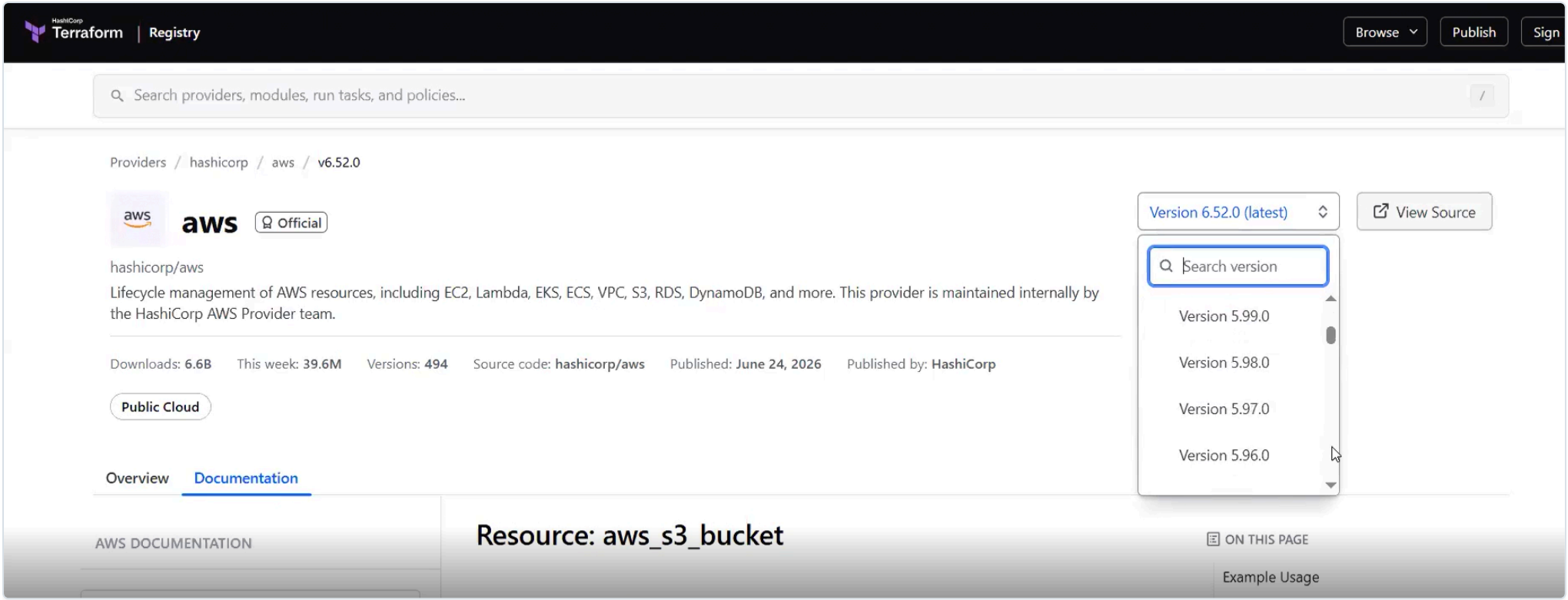

terraform {
required_providers {
aws = {
version = "~> 5.52.0"
}
random = {
version = "~> 3.6.2"
}
}
required_version = "~> 1.1.9"
}

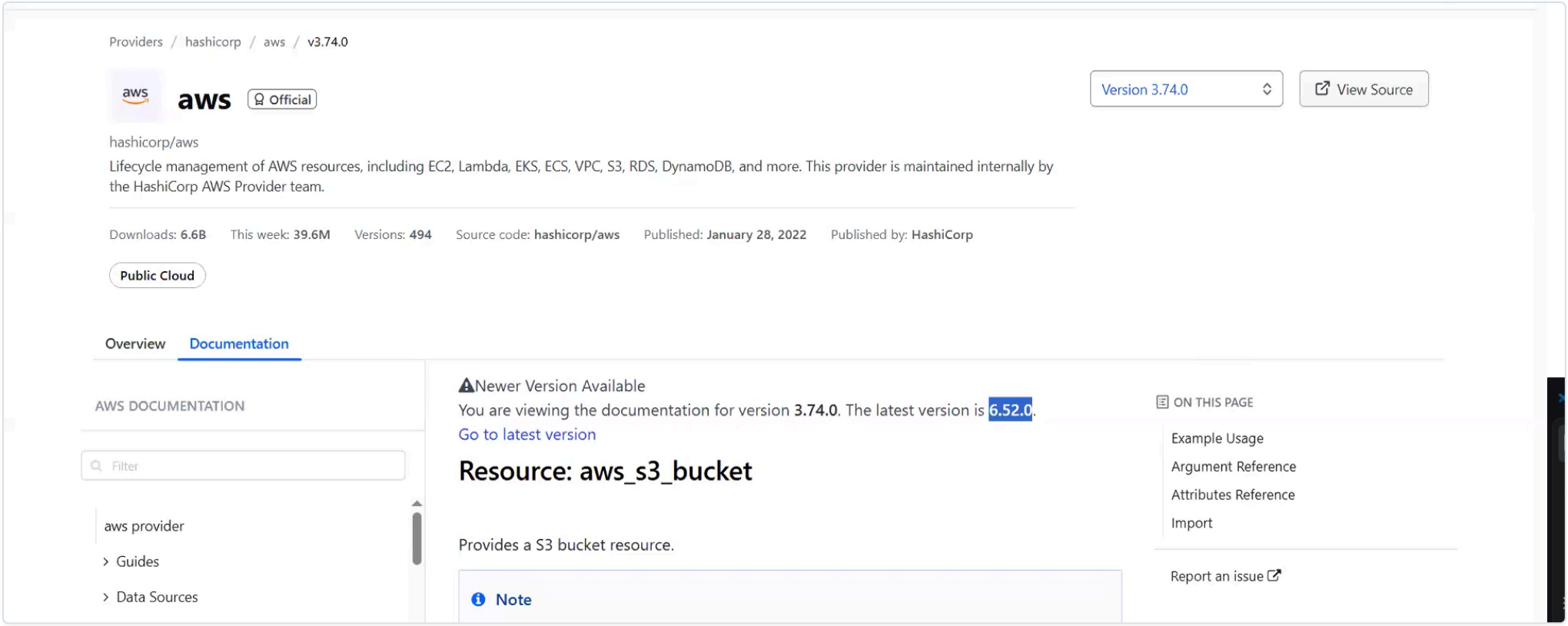
Required Version	Meaning	Considerations
`1.7.5`	Only Terraform v1.7.5 exactly	To upgrade Terraform, first edit the `required_version` setting
`>= 1.7.5`	Any Terraform v1.7.5 or greater	Includes Terraform v2.0.0 and above
`~> 1.7.5`	Any Terraform v1.7.x, but not v1.8 or later	Patch version updates should be non-disruptive
`>= 1.7.1, < 1.9.5`	Terraform v1.7.5 or greater, but less than v1.9.5	Avoids specific version updates


[root@Devops-server ~]# mkdir a b
[root@Devops-server ~]# cd a/
[root@Devops-server a]#


HashiCorp
Terraform | Registry
Search providers, modules, run tasks, and policies...
Providers / hashicorp / aws / v6.52.0
aws
aws
Official
Vers
hashicorp/aws
Lifecycle management of AWS resources, including EC2, Lambda, EKS, ECS, VPC, S3, RDS, DynamoDB, and more. This provider is maintained internally by the HashiCorp AWS Provider team.
Downloads: 6.6B This week: 39.6M Versions: 494 Source code: hashicorp/aws Published: June 24, 2026 Published by: HashiCorp
Public Cloud
Overview Documentation
AWS DOCUMENTATION
Resource: aws_s3_bucket

Version picker — 494 versions




Providers / hashicorp / aws / v3.74.0
aws
Official
Version 3.74.0
View Source
hashicorp/aws
Lifecycle management of AWS resources, including EC2, Lambda, EKS, ECS, VPC, S3, RDS, DynamoDB, and more. This provider is maintained internally by the HashiCorp AWS Provider team.
Downloads: 6.6B This week: 39.6M Versions: 494 Source code: hashicorp/aws Published: January 28, 2022 Published by: HashiCorp
Public Cloud
Overview
Documentation
AWS DOCUMENTATION
Filter
aws provider
Guides
Data Sources
Newer Version Available
You are viewing the documentation for version 3.74.0. The latest version is 6.52.0.
Go to latest version
Resource: aws_s3_bucket
Provides a S3 bucket resource.
Note
ON THIS PAGE
Example Usage
Argument Reference
Attributes Reference
Import
Report an issue

Example Usage

Report an

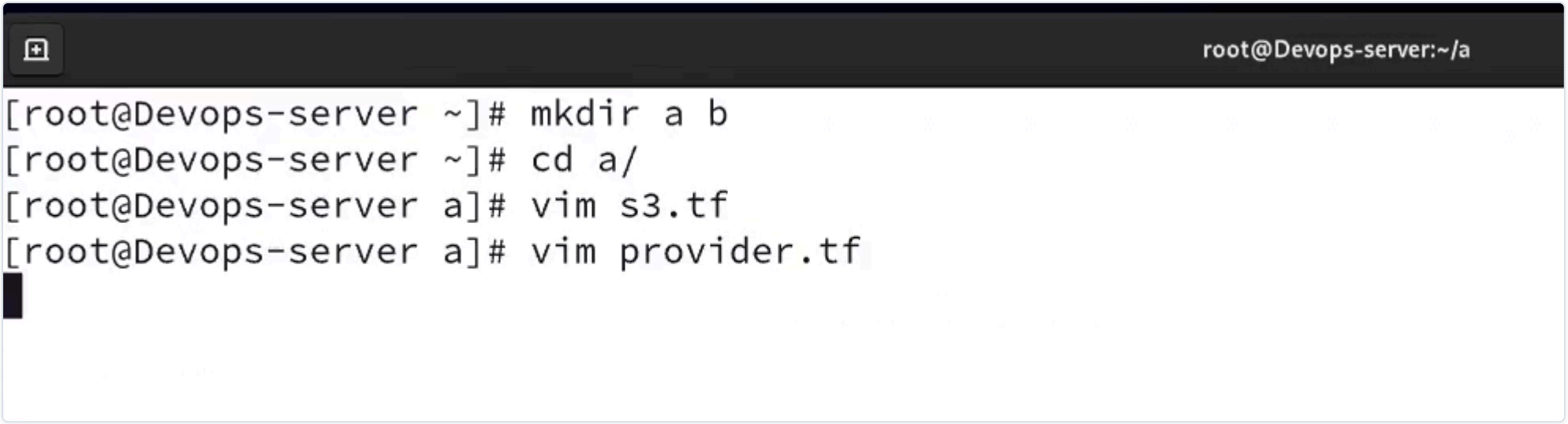
Private Bucket w/ Tags

Copied!

```
resource "aws_s3_bucket" "b" {
  bucket = "my-tf-test-bucket"
  acl    = "private"

  tags = {
    Name        = "My bucket"
    Environment = "Dev"
  }
}
```


root@Devops-server:~/a — vim s3.tf
1 resource "aws_s3_bucket" "b" {
2 bucket = "my-tf-test-bucket"
3 acl = "private"
4
5 tags = {
6 Name = "My bucket"
7 Environment = "Dev"
8 }
9 }


root@Devops-server:~/a
[root@Devops-server ~]# mkdir a b
[root@Devops-server ~]# cd a/
[root@Devops-server a]# vim s3.tf
[root@Devops-server a]# vim provider.tf

root@Devops-server:~/a

```
[root@Devops-server a]# nl provider.tf
     1  terraform {
     2    required_providers {
     3      aws = {
     4        source = "hashicorp/aws"
     5        version = "~>3.74.0"
     6      }
     7    }
     8  }

[root@Devops-server a]# nl s3.tf
     1  resource "aws_s3_bucket" "b" {
     2    bucket = "my-tf-test-bucket"
     3    acl    = "private"

     4    tags = {
     5      Name        = "My bucket"
     6      Environment = "Dev"
     7    }
     8  }
[root@Devops-server a]#
```

```
[root@Devops-server a]# terraform init
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 3.74.0"...
- Installing hashicorp/aws v3.74.3...
```

```
[root@Devops-server a]# terraform init
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 3.74.0"...
- Installing hashicorp/aws v3.74.3...
- Installed hashicorp/aws v3.74.3 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

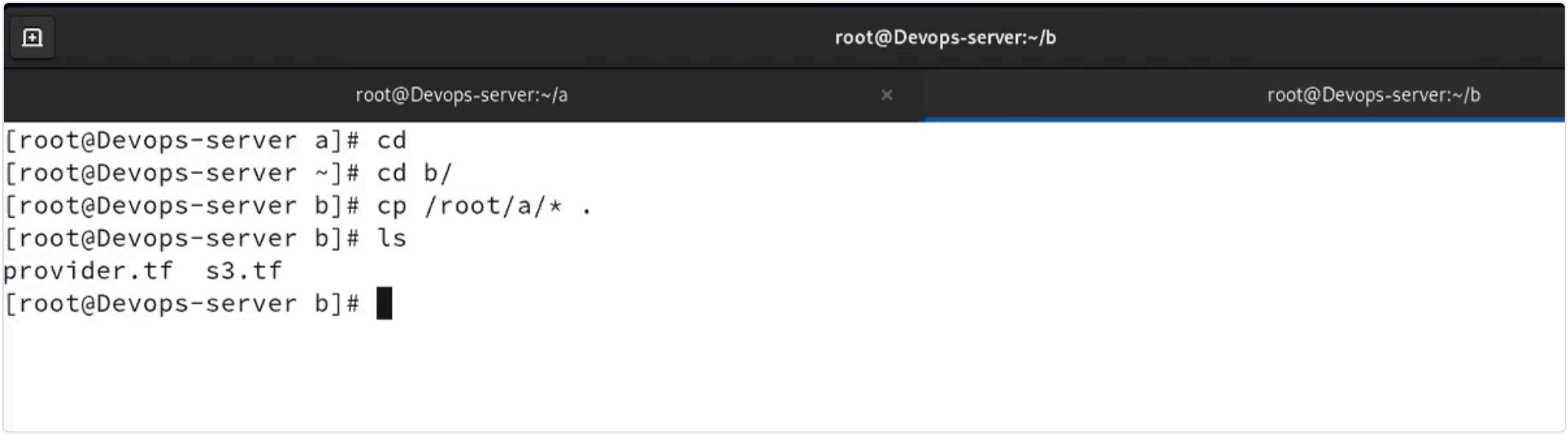
You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
[root@Devops-server a]#
```

root@Devops-server:~/a

```
[root@Devops-server a]# cat .terraform.lock.hcl
# This file is maintained automatically by "terraform init".
# Manual edits may be lost in future updates.

provider "registry.terraform.io/hashicorp/aws" {
  version     = "3.74.3"
  constraints = "~> 3.74.0"
  hashes = [
    "h1:h4TYqgRKTuuWfZtxJnEGcs/NxGCaxZ4jr0IwTfgZDRM=",
    "zh:25401cd4667d0496caf7e92e74ecef7c98cf74465570705cda2207770c27ff6c",
    "zh:2d154527a9b2585f72fc5eceac635257e3f50f68de8a519e71c795d5166a0a22",
    "zh:499fa5201804a5a33a90d683147fb2f81da91bfcd8ed20293f88f6f39cedbf97",
    "zh:730284250fd949a59afb6935b3a68a33709d5a78b686fa98f351ad32c919cfc3",
    "zh:7461ebd6fb35900d620cfa3f42126d988ea1e604ee3828d1c64d5727f908bd26",
    "zh:7c85743b31c7459f8e74aaa98471ba82c54517eb908603411808a12982d89b1c",
    "zh:8ed977b7fb97de624f5414b08cab36fd973a624072e0e9082c0c822e0864c7b9",
    "zh:94ae7313bb0b425d4007a0b70601a337972c4f0f7a323487acf69215e74b4425",
    "zh:b5a1589672d709da725a72c46d28bf5b2dea71325f6e0b44a0049f644cd09eba",
    "zh:c7e8e7ce59e4578416557fc2f138137af3c8365ac3e34f0ff5166323c7d641a1",
    "zh:ccf2e286b207e749fff76bb4075deddb9e7e237936d8654f34828c54e7035455",
  ]
}
```


root@Devops-server:~/b
root@Devops-server:~/a
root@Devops-server:~/b
[root@Devops-server a]# cd
[root@Devops-server ~]# cd b/
[root@Devops-server b]# cp /root/a/* .
[root@Devops-server b]# ls
provider.tf s3.tf
[root@Devops-server b]#


root@Devops-server:~/a
[root@Devops-server a]# cd
[root@Devops-server ~]# cd b/
[root@Devops-server b]# cp /root/a/* .
[root@Devops-server b]# ls
provider.tf s3.tf
[root@Devops-server b]# rm -rf provider.tf
[root@Devops-server b]#

```
[root@Devops-server b]# cat s3.tf
resource "aws_s3_bucket" "b" {
  bucket = "my-tf-test-bucket"
  acl    = "private"

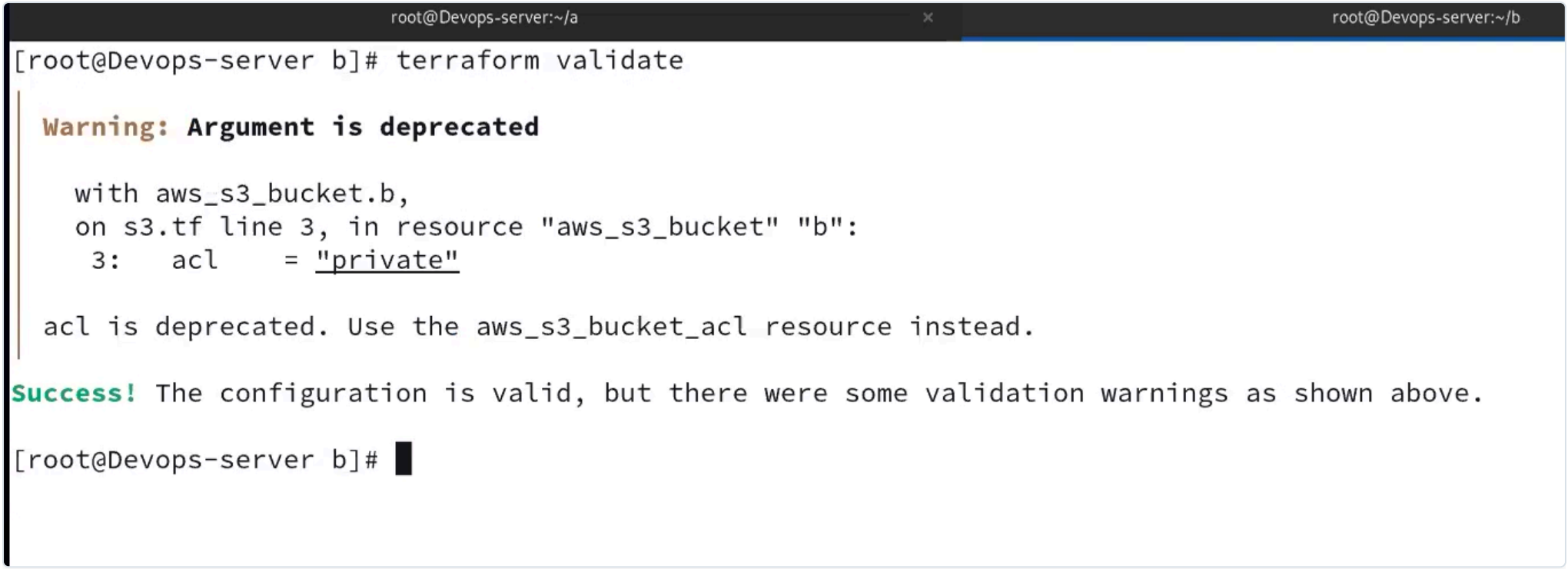
  tags = {
    Name        = "My bucket"
    Environment = "Dev"
  }
}
[root@Devops-server b]# terraform init
Initializing the backend...
Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.52.0...
```

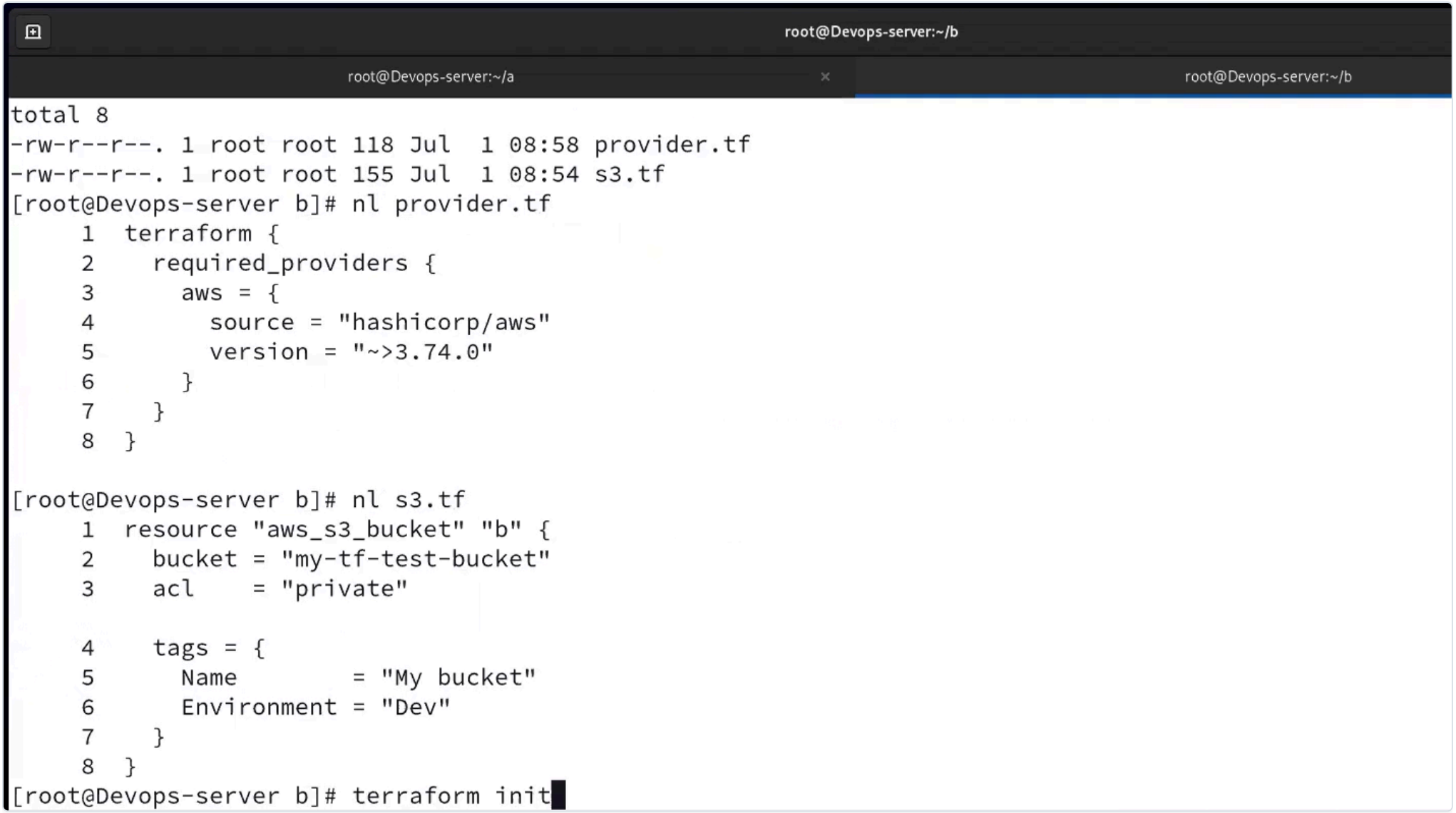
```
[root@Devops-server b]# terraform init
Initializing the backend...
Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.52.0...
- Installed hashicorp/aws v6.52.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
[root@Devops-server b]#
```


root@Devops-server:~/a
root@Devops-server:~/b
[root@Devops-server b]# terraform validate
Warning: Argument is deprecated
with aws_s3_bucket.b,
on s3.tf line 3, in resource "aws_s3_bucket" "b":
3: acl = "private"
acl is deprecated. Use the aws_s3_bucket_acl resource instead.
Success! The configuration is valid, but there were some validation warnings as shown above.
[root@Devops-server b]#


root@Devops-server:~/b
root@Devops-server:~/a
root@Devops-server:~/b
total 8
-rw-r--r--. 1 root root 118 Jul 1 08:58 provider.tf
-rw-r--r--. 1 root root 155 Jul 1 08:54 s3.tf
[root@Devops-server b]# nl provider.tf
1 terraform {
2 required_providers {
3 aws = {
4 source = "hashicorp/aws"
5 version = "~>3.74.0"
6 }
7 }
8 }
[root@Devops-server b]# nl s3.tf
1 resource "aws_s3_bucket" "b" {
2 bucket = "my-tf-test-bucket"
3 acl = "private"
4 tags = {
5 Name = "My bucket"
6 Environment = "Dev"
7 }
8 }
[root@Devops-server b]# terraform init

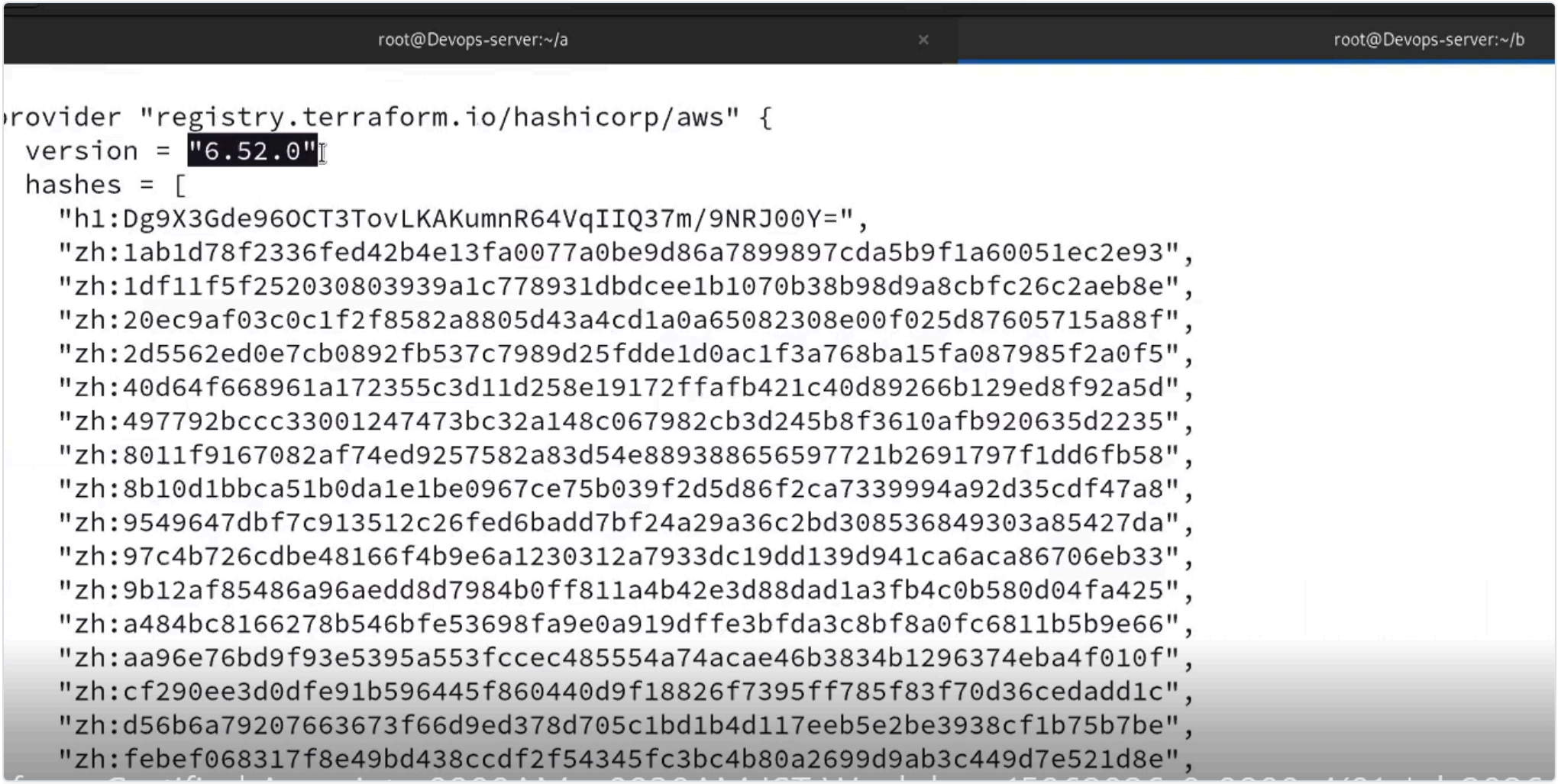
```
[root@Devops-server b]# terraform init
Initializing the backend...
Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file

  Error: Failed to query available provider packages

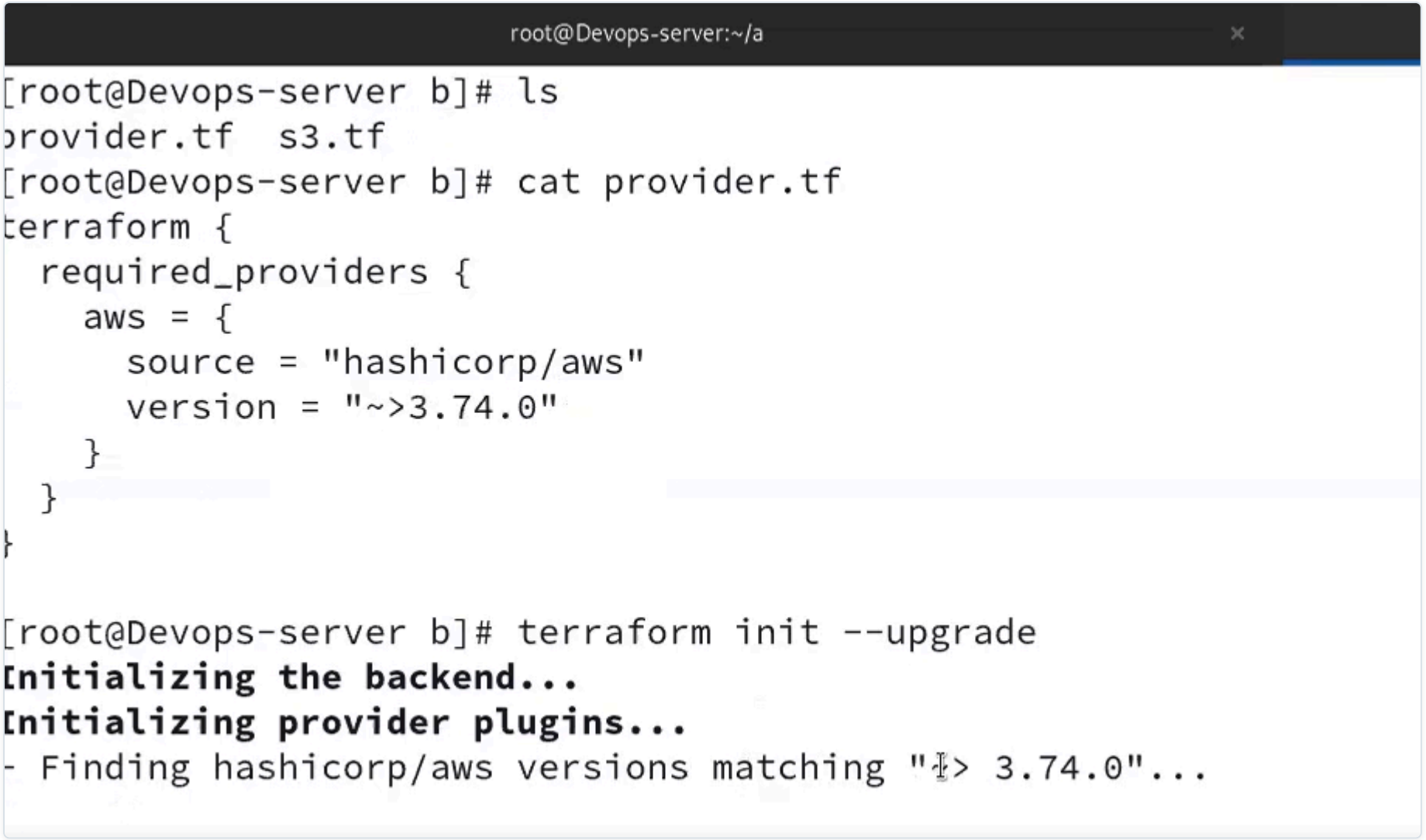
  Could not retrieve the list of available versions for provider hashicorp/aws: locked provider
  registry.terraform.io/hashicorp/aws 6.52.0 does not match configured version constraint ~> 3.74.0; must use terraform
  init -upgrade to allow selection of new versions

  To see which modules are currently depending on hashicorp/aws and what versions are specified, run the following
  command:
      terraform providers

[root@Devops-server b]#
```


root@Devops-server:~/a
root@Devops-server:~/b
provider "registry.terraform.io/hashicorp/aws" {
version = "6.52.0"
hashes = [
"h1:Dg9X3Gde96OCT3TovLKAKumnR64VqIIQ37m/9NRJ00Y=",
"zh:1ab1d78f2336fed42b4e13fa0077a0be9d86a7899897cda5b9f1a60051ec2e93",
"zh:1df11f5f252030803939a1c778931dbdcee1b1070b38b98d9a8cbfc26c2aeb8e",
"zh:20ec9af03c0c1f2f8582a8805d43a4cd1a0a65082308e00f025d87605715a88f",
"zh:2d5562ed0e7cb0892fb537c7989d25fdde1d0ac1f3a768ba15fa087985f2a0f5",
"zh:40d64f668961a172355c3d11d258e19172ffafb421c40d89266b129ed8f92a5d",
"zh:497792bccc33001247473bc32a148c067982cb3d245b8f3610afb920635d2235",
"zh:8011f9167082af74ed9257582a83d54e889388656597721b2691797f1dd6fb58",
"zh:8b10d1bbca51b0da1e1be0967ce75b039f2d5d86f2ca7339994a92d35cdf47a8",
"zh:9549647dbf7c913512c26fed6badd7bf24a29a36c2bd308536849303a85427da",
"zh:97c4b726cdbe48166f4b9e6a1230312a7933dc19dd139d941ca6aca86706eb33",
"zh:9b12af85486a96aedd8d7984b0ff811a4b42e3d88dad1a3fb4c0b580d04fa425",
"zh:a484bc8166278b546bfe53698fa9e0a919dffe3bfda3c8bf8a0fc6811b5b9e66",
"zh:aa96e76bd9f93e5395a553fccec485554a74acae46b3834b1296374eba4f010f",
"zh:cf290ee3d0dfe91b596445f860440d9f18826f7395ff785f83f70d36cedadd1c",
"zh:d56b6a79207663673f66d9ed378d705c1bd1b4d117eeb5e2be3938cf1b75b7be",
"zh:febef068317f8e49bd438ccdf2f54345fc3bc4b80a2699d9ab3c449d7e521d8e",

```
[root@Devops-server b]# terraform init --upgrade
```


root@Devops-server:~/a
[root@Devops-server b]# ls
provider.tf s3.tf
[root@Devops-server b]# cat provider.tf
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~>3.74.0"
    }
  }
}
[root@Devops-server b]# terraform init --upgrade
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 3.74.0"...


root@Devops-server:~/b
root@Devops-server:~/a
[root@Devops-server b]# terraform validate
Success! The configuration is valid.
[root@Devops-server b]#

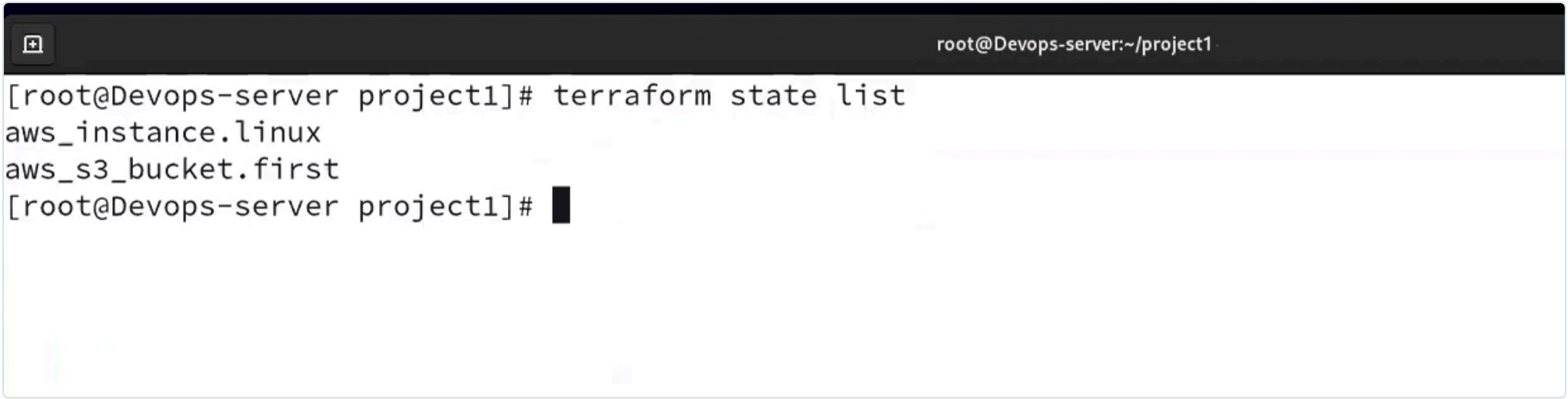
root@Devops-server:~/project1 — vim main.tf

```
########## Launch Ec2 Instance in Mumbai region
resource "aws_instance" "linux" {
  ami           = "ami-0cca134ec43cf708f"
  instance_type = "t2.micro"
  key_name      = "ram_aws"
  tags = {
    Name = "Test_server"
    source = "terraform"
  }
}

########## Launch S3 Bucket in Mumbai region
resource "aws_s3_bucket" "first" {
  bucket = "linux23212google123"
  tags = {
        Name = "my_buck"
}
}

```

The `terraform state rm` command removes a resource from your Terraform state file without destroying the actual infrastructure. Once removed, Terraform stops tracking the infrastructure object, leaving it fully intact in your cloud provider. HashiCorp Developer +2


root@Devops-server:~/project1
[root@Devops-server project1]# terraform state list
aws_instance.linux
aws_s3_bucket.first
[root@Devops-server project1]#


root@Devops-server:~/project1
[root@Devops-server project1]# terraform state list
aws_instance.linux
aws_s3_bucket.first
[root@Devops-server project1]# terraform state rm aws_s3_bucket.first
Removed aws_s3_bucket.first
Successfully removed 1 resource instance(s).
[root@Devops-server project1]# terraform state list
aws_instance.linux
[root@Devops-server project1]#

The **`terraform state mv`** command renames or relocates resources within your Terraform state file without destroying or recreating the actual infrastructure. It updates the mapping between your updated code configuration and the existing cloud assets. OneUptime +1

```
aws_s3_bucket.first
[root@Devops-server project1]# terraform state mv aws_s3_bucket.first aws_s3_bucket.hello
```